

Null Handling of Repository Methods

Null Handling of Repository Methods

Nullability Annotations

[JSpecify](#)

[Spring Framework Nullability and JSR-305 Annotations](#)

[Nullability in Kotlin-based Repositories](#)

Repository CRUD methods that return an individual aggregate instances can use `Optional` to indicate the potential absence of a value. Besides that, Spring Data supports returning the following wrapper types on query methods:

- `com.google.common.base.Optional`
- `scala.Option`
- `io.vavr.control.Option`

Alternatively, query methods can choose not to use a wrapper type at all. The absence of a query result is then indicated by returning `null`. Repository methods returning collections, collection alternatives, wrappers, and streams are guaranteed never to return `null` but rather the corresponding empty representation. See “[Repository query return types](#)” for details.

Nullability Annotations

JSpecify

As on Spring Framework 7 and Spring Data 4, you can express nullability constraints for repository methods by using [JSpecify](#). JSpecify is well integrated into IntelliJ and Eclipse to provide a tooling-friendly approach and opt-in `null` checks during runtime, as follows:

- [@NotNullMarked](#): Used on the module-, package- and class-level to declare that the default behavior for parameters and return values is, respectively, neither to accept nor to produce `null` values.
- [@Nonnull](#): Used on a type level for parameter or return values that must not be `null` (not needed value where [@NotNullMarked](#) applies).

- [@Nullable](#): Used on the type level for parameter or return values that can be null.
- [@NullUnmarked](#): Used on the package-, class-, and method-level to roll back nullness declaration and opt-out from a previous [@Nullable](#). Nullness changes to unspecified in such a case.

@Nullable at the package level via a package-info.java file

```
@Nullable
package org.springframework.core;

import org.jspecify.annotations.Nullable;
```

JAVA

In the various Java files belonging to the package, nullable type usages are defined explicitly with [@Nullable](#). It is recommended that this annotation is specified just before the related type.

For example, for a field:

```
private @Nullable String fileEncoding;
```

JAVA

Or for method parameters and return value:

```
public static @Nullable String buildMessage(@Nullable String message,
                                           @Nullable Throwable cause) {
    // ...
}
```

JAVA

When overriding a method, nullness annotations are not inherited from the superclass method. That means those nullness annotations should be repeated if you just want to override the implementation and keep the same API nullness.

With arrays and varargs, you need to be able to differentiate the nullness of the elements from the nullness of the array itself. Pay attention to the syntax defined by the Java specification which may be initially surprising:

- `@Nullable Object[] array` means individual elements can be null but the array itself can't.
- `Object @Nullable [] array` means individual elements can't be null but the array itself can.
- `@Nullable Object @Nullable [] array` means both individual elements and the array can be null.

The Java specifications also enforces that annotations defined with `@Target(ElementType.TYPE_USE)` like JSpecify `@Nullable` should be specified after the last `.` with inner or fully qualified types:

- `Cache.@Nullable ValueWrapper`

- `jakarta.validation.@Nullable` Validator

[@NonNull](#) and [@NotNullUnmarked](#) should rarely be needed for typical use cases.

Spring Framework Nullability and JSR-305 Annotations

You can express nullability constraints for repository methods by using [Spring Framework's nullability annotations](#).

NOTE

As on Spring Framework 7, Spring's nullability annotations are deprecated in favor of JSpecify. Consult the framework documentation on [Migrating from Spring null-safety annotations to JSpecify](#) for more information.

They provide a tooling-friendly approach and opt-in `null` checks during runtime, as follows:

- [@NonNullApi](#) : Used on the package level to declare that the default behavior for parameters and return values is, respectively, neither to accept nor to produce `null` values.
- [@NonNull](#) : Used on a parameter or return value that must not be `null` (not needed on a parameter and return value where [@NonNullApi](#) applies).
- [@Nullable](#) : Used on a parameter or return value that can be `null`.

Spring annotations are meta-annotated with [JSR 305](#) annotations (a dormant but widely used JSR). JSR 305 meta-annotations let tooling vendors (such as [IDEA](#), [Eclipse](#), and [Kotlin](#)) provide null-safety support in a generic way, without having to hard-code support for Spring annotations. To enable runtime checking of nullability constraints for query methods, you need to activate non-nullability on the package level by using Spring's [@NonNullApi](#) in `package-info.java`, as shown in the following example:

Declaring Non-nullability in package-info.java

JAVA

Once non-null defaulting is in place, repository query method invocations get validated at runtime for nullability constraints. If a query result violates the defined constraint, an exception is thrown. This happens when the method would return `null` but is declared as non-nullable (the default with the annotation defined on the package in which the repository resides). If you want to opt-in to nullable results again, selectively use [@Nullable](#) on individual methods. Using the result wrapper types mentioned at the start of this section continues to work as expected: an empty result is translated into the value that represents absence.

The following example shows a number of the techniques just described:

Using different nullability constraints

```

package com.acme; 1

interface UserRepository extends Repository<User, Long> {

    User getByEmailAddress(EmailAddress emailAddress); 2

    @Nullable
    User findByEmailAddress(@Nullable EmailAddress emailAddress); 3

    Optional<User> findOptionalByEmailAddress(EmailAddress emailAddress); 4
}

```

- 1 The repository resides in a package (or sub-package) for which we have defined non-null behavior.
- 2 Throws an `EmptyResultDataAccessException` when the query does not produce a result. Throws an `IllegalArgumentException` when the `emailAddress` handed to the method is `null`.
- 3 Returns `null` when the query does not produce a result. Also accepts `null` as the value for `emailAddress`.
- 4 Returns `Optional.empty()` when the query does not produce a result. Throws an `IllegalArgumentException` when the `emailAddress` handed to the method is `null`.

Nullability in Kotlin-based Repositories

Kotlin has the definition of [nullability constraints](#) baked into the language. Kotlin code compiles to byte-code, which does not express nullability constraints through method signatures but rather through compiled-in metadata. Make sure to include the `kotlin-reflect` JAR in your project to enable introspection of Kotlin's nullability constraints. Spring Data repositories use the language mechanism to define those constraints to apply the same runtime checks, as follows:

Using nullability constraints on Kotlin repositories

```

interface UserRepository : Repository<User, String> {

    fun findByUsername(username: String): User 1

    fun findByFirstname(firstname: String?): User? 2
}

```

- 1 The method defines both the parameter and the result as non-nullable (the Kotlin default). The Kotlin compiler rejects method invocations that pass `null` to the method. If the query yields an empty result, an `EmptyResultDataAccessException` is thrown.
- 2 This method accepts `null` for the `firstname` parameter and returns `null` if the query does not produce a result.

Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.